`SingletonMessageService.php` file contains another implementation of the `MessageServiceInterface`. This service will be bound as a singleton service.

```
<?php

namespace App\Services;

use App\Services\MessageServiceInterface;

class SingletonMessageService implements MessageServiceInterface
{
    public string $timestamp;

    public function __construct()
    {
        $this->timestamp = now();
    }

    public function getMessage(): string
    {
        return "Hello from SingletonMessageService at {$this->timestamp}";
    }
}
```

Save all three files.

5.11.4.3 Step 3: Register Bindings in AppServiceProvider

Open `app/Providers/AppServiceProvider.php` and add this to the `register()` method:

```
use App\Services\MessageServiceInterface;
use App\Services\StandardMessageService;
use App\Services\SingletonMessageService;

public function register(): void
{
    // Transient (default) binding
    $this->app->bind('standard.message', StandardMessageService::class);

    // Singleton binding
    $this->app->singleton('singleton.message', SingletonMessageService::class);

    // Interface binding (default uses Standard)
    $this->app->bind(MessageServiceInterface::class, StandardMessageService::class);
}
```

5.11.4.4 Step 4: Create a Controller with Different Injection Types

Now we will create a controller that uses both constructor and method injection.

```
php artisan make:controller MessageController
```